E-Health Insurance Platform

Technical Specification & Architecture Document

This document describes the architecture, services, APIs, and data model of the E-Health Insurance Platform — a microservices-based system that lets members buy and manage health insurance policies online, submit medical claims, and receive automated reimbursement decisions, while giving insurance staff tools to manage plans and review flagged claims.

Document scope

- Project overview and user journey
- Architecture and tech stack
- Repository structure
- Per-service specification (purpose, flow, APIs, entities)
- Cross-cutting concerns (authentication, local dev, future work)

1. Project overview

1.1 What the platform does

E-Health Insurance is a digital platform that runs the full lifecycle of health insurance for both members and the insurance company. Members sign up, choose a plan, submit claims when they receive medical care, and get reimbursed automatically for covered procedures. The insurance company's staff manage the plan catalog, review flagged claims, and use AI-assisted tools to detect fraud and answer member questions.

The system is built as six microservices behind an API gateway. Each service owns its own database (database-per-service pattern), communicates with other services via REST, and can be developed and deployed independently. The architecture supports scaling each service separately as load grows and lets teams work on different parts of the system in parallel.

1.2 User roles

The platform supports three distinct user types, each with different permissions enforced through role-based access control (RBAC):

- **Members** — end users who buy insurance, submit claims, track claim status, and interact with the AI chatbot. Can read and update their own data only.
- **Staff (claim reviewers)** — insurance company employees who manually review flagged claims, approve or reject them, and respond to member disputes. Can read and update any claim and view all member policies.
- **Administrators** — manage the plan catalog, create new plans, adjust pricing, manage staff accounts, and view system analytics.

1.3 Core user journey

The platform supports an end-to-end insurance lifecycle:

1. **Signup and plan selection** — Member creates an account through the IAM service, optionally requests a plan recommendation from the AI service, and selects a plan in the Policy service.
2. **Medical care occurs** — This happens outside the system. Member receives treatment and accumulates bills from providers.
3. **Claim submission** — Member uploads bills and supporting evidence through the Claim service. The AI service may extract structured data from the documents.
4. **Automated review** — The Claim service consults the Policy service for coverage rules and the AI service for fraud scoring. Based on the combined verdict, the claim is auto-approved, auto-rejected, or routed to a human reviewer.
5. **Notification** — The Notification service emails the member with the decision.
6. **Ongoing support** — Members can chat with the AI assistant any time for coverage questions, claim status updates, and plan information.

1.4 Tech stack

- **Backend:** Java 21, Spring Boot 3.x
- **Build tool:** Maven
- **Database:** PostgreSQL 16 (one database per service)
- **Migrations:** Flyway
- **Inter-service calls:** REST via OpenFeign clients
- **Email (dev):** MailHog (catches outbound mail locally)
- **Containerization:** Docker and docker-compose
- **Source control:** Git, hosted on GitLab
- **External APIs:** LLM provider (Anthropic or OpenAI) for AI service capabilities

1.5 Service overview

Service	Port	Responsibility
Gateway	8080	API entry point, routing, JWT validation
IAM	8081	Authentication, JWT, profiles, roles
Policy	8082	Plans, coverage, eligibility
Claim	8083	Claims, decisions, evidence
AI	8084	Scoring, chat, recommendations
Notification	8085	Email and SMS delivery

2. Repository structure

The project consists of seven Git repositories hosted under one GitLab group called *e-health-insurance*. Six are application services (each a Spring Boot project) and one is shared infrastructure (docker-compose, init scripts, this document).

Repository	Purpose
e-health-insurance-gw	API gateway
e-health-insurance-iam	Identity and access management
e-health-insurance-policy	Plans and eligibility
e-health-insurance-claim	Claims and evidence
e-health-insurance-ai	AI capabilities
e-health-insurance-notification	Outbound messages
e-health-insurance-infra	Docker-compose, scripts, shared docs

2.1 Developer workspace

Each developer clones all seven repositories as siblings into one local workspace folder. The infra repo contains the docker-compose.yml that references each service via relative path (`../e-health-insurance-iam`, etc.). To start the whole system locally, a developer runs `docker-compose up -d` from inside the infra folder.

3. Services

3.1 Gateway service

Purpose

The Gateway is the single entry point for all client traffic (web frontend, mobile app). It routes incoming requests to the appropriate downstream service based on URL path prefix, validates JWT tokens before forwarding, and handles cross-cutting concerns like rate limiting and CORS. The Gateway is pure infrastructure — it does not own any business data and has no database.

Internal flow

1. Receive HTTP request from client.
2. Validate the JWT signature and expiry.
3. Apply rate limiting and CORS headers.
4. Identify the target service from the URL path prefix.
5. Forward the request to the target service, preserving headers.
6. Return the downstream response to the client.

Inter-service flow

- **Calls:** all five business services (IAM, Policy, Claim, AI, Notification)
- **Called by:** external clients only (web frontend, mobile app)

APIs (routing rules, not business endpoints)

Method	Path	Description
`ANY`	`/api/auth/* /api/users/*`	Forwarded to IAM service
`ANY`	`/api/policies/*`	Forwarded to Policy service
`ANY`	`/api/claims/* /api/evidence/*`	Forwarded to Claim service
`ANY`	`/api/ai/* /api/chat/*`	Forwarded to AI service
`ANY`	`/api/notifications/*`	Forwarded to Notification service
`GET`	`/actuator/health`	Gateway health check

Entities

None. The gateway is stateless and owns no data.

3.2 IAM service

Purpose

The IAM (Identity and Access Management) service is the source of truth for user identity, authentication, and authorization across the entire platform. It manages user registration, verifies credentials, issues JWT tokens on successful login, validates tokens for other services on request, and stores user profiles with assigned roles. It implements role-based access control with three roles: MEMBER, STAFF, and ADMIN.

Internal flow — registration

1. Receive signup data (email, password, profile fields).
2. Validate email uniqueness and password strength.
3. Hash the password using BCrypt.
4. Create a user record with the default MEMBER role.
5. Trigger a welcome notification via the Notification service.
6. Return the new user's ID (without auto-login).

Internal flow — login

1. Receive credentials (email and password).
2. Look up the user by email.
3. Verify the password against the stored BCrypt hash.
4. Generate a JWT with user_id, role, and expiry claims.
5. Update last_login_at timestamp.
6. Return the JWT to the client.

Inter-service flow

- **Calls:** Notification service (welcome emails, password reset emails)
- **Called by:** Gateway (auth endpoints), all other services (JWT validation, profile lookup)

APIs

Method	Path	Description
`POST`	`/api/auth/register`	Create a new user account
`POST`	`/api/auth/login`	Authenticate and return JWT
`POST`	`/api/auth/refresh`	Refresh an expired JWT
`POST`	`/api/auth/validate`	Validate a JWT (called by other services)
`GET`	`/api/users/me`	Get the current user's profile
`PUT`	`/api/users/me`	Update the current user's profile

GET	/api/users/{id}	Get a user by ID (staff or admin only)
PATCH	/api/users/{id}/role	Change a user's role (admin only)

Entities

user

Field	Description
id	Unique user identifier (UUID)
email	Login email, unique
password_hash	BCrypt hash of the password
first_name	Given name
last_name	Family name
phone	Phone number for SMS notifications
role	MEMBER, STAFF, or ADMIN
status	ACTIVE or DISABLED
created_at	Registration timestamp
last_login_at	Timestamp of last successful login

3.3 Policy service

Purpose

The Policy service manages the catalog of insurance plans (defined by administrators) and individual member policies (one per member, each pointing to a plan). It answers eligibility and coverage questions for the Claim service during claim adjudication. The plan catalog is the source of truth for what products the insurance company offers; member policies record who has purchased what.

Internal flow — member buys a plan

1. Receive plan purchase request (plan_id, member_id, start_date).
2. Validate that the plan exists and is currently active.
3. Create a policy record with start and end dates.
4. Return policy details to the caller.

Internal flow — coverage check

1. Receive coverage query (member_id, procedure_code, claim_amount).
2. Look up the member's active policy.
3. Apply plan rules: is the procedure covered, what's the coverage percentage, has the deductible been met, are there yearly limits.
4. Return verdict (covered yes/no) and the calculated reimbursement amount.

Inter-service flow

- **Calls:** none
- **Called by:** Claim service (coverage checks during adjudication), AI service (plan catalog for recommendations)

APIs

Method	Path	Description
`GET`	`/api/policies/plans`	List all available plans
`GET`	`/api/policies/plans/{id}`	Get plan details by ID
`POST`	`/api/policies/plans`	Create a new plan (admin only)
`PUT`	`/api/policies/plans/{id}`	Update a plan (admin only)
`POST`	`/api/policies/purchase`	Buy a plan for a member
`GET`	`/api/policies/me`	Get the current member's active policy
`GET`	`/api/policies/{id}/coverage`	Check coverage for a procedure

Entities

plan

Field	Description
`id`	Unique plan identifier
`name`	Plan name (e.g. "Standard Plus")
`description`	Plan description for members
`monthly_premium`	Monthly cost in currency
`coverage_percent`	Reimbursement rate (e.g. 0.80 = 80%)
`annual_limit`	Maximum yearly reimbursement
`deductible`	Amount the member pays before coverage starts
`covered_procedures`	List of covered procedure codes
`status`	ACTIVE or RETIRED
`created_at`	Creation timestamp

policy

Field	Description
`id`	Unique policy identifier
`member_id`	Member who owns this policy (from IAM)
`plan_id`	Which plan was purchased
`start_date`	Policy start date
`end_date`	Policy end date
`status`	ACTIVE, EXPIRED, or CANCELLED
`deductible_paid`	Deductible already paid this period
`claims_total`	Total reimbursed this period
`created_at`	Purchase timestamp

3.4 Claim service

Purpose

The Claim service is the heart of the platform. It manages claim submission, stores supporting evidence (medical bills, prescriptions, reports), coordinates the adjudication decision by consulting the Policy and AI services, and records the outcome. It decides whether each claim is auto-approved, auto-rejected, or sent to a human reviewer based on the combined coverage and fraud-score signals it receives.

Internal flow — claim submission

1. Receive a new claim from the member (amount, procedure code, diagnosis code, evidence references).
2. Store the claim with status PENDING.
3. Call the Policy service to check coverage.
4. Call the AI service to score for fraud.
5. Combine the two verdicts: if coverage approves and fraud score is low, auto-approve; if fraud score is high, route to manual review; if not covered, auto-reject.
6. Update the claim status and store the decision reason.
7. Trigger a notification to the member via the Notification service.

Internal flow — evidence upload

1. Member uploads a file (bill, prescription, or other document).
2. File is stored to a local volume (planned: cloud storage in production).
3. An evidence record is created linking the file to a claim.
4. The AI service may be called to OCR the file and extract structured data.

Inter-service flow

- **Calls:** Policy (coverage checks), AI (fraud scoring and OCR), Notification (decision emails)
- **Called by:** Gateway (member submissions), AI (claim history lookups)

APIs

Method	Path	Description
`POST`	`/api/claims`	Submit a new claim
`GET`	`/api/claims/me`	List the current member's claims
`GET`	`/api/claims/{id}`	Get claim details
`POST`	`/api/claims/{id}/evidence`	Upload a supporting document
`GET`	`/api/claims/{id}/evidence`	List evidence for a claim

PATCH	/api/claims/{id}/review	Manual decision by staff
GET	/api/claims/flagged	List claims awaiting manual review (staff only)

Entities

claim

Field	Description
id	Unique claim identifier
member_id	Claimant (from IAM)
policy_id	Under which policy the claim is filed
amount	Claimed amount
procedure_code	What procedure was performed
diagnosis_code	What diagnosis prompted the procedure
provider_name	Who provided the care
service_date	When the care was given
status	PENDING, APPROVED, REJECTED, or MANUAL_REVIEW
decision_reason	Why this decision was made
approved_amount	Actual reimbursement amount
submitted_at	Submission timestamp
decided_at	Decision timestamp

evidence

Field	Description
id	Unique evidence identifier
claim_id	Which claim this supports
file_path	Where the file is stored
file_type	BILL, PRESCRIPTION, REPORT, or OTHER
file_size	Size in bytes
uploaded_at	Upload timestamp

3.5 AI service

Purpose

The AI service provides intelligent capabilities to the platform: fraud and risk scoring on submitted claims, document OCR for evidence extraction, plain-language decision explanations, a member-facing chatbot, and plan recommendations at signup. It calls external LLM and OCR APIs rather than running ML models locally — it is primarily an integration and orchestration layer for AI capabilities.

Internal flow — claim scoring

1. Receive claim data (amount, procedure, member, evidence references).
2. Fetch the member's recent claim history from the Claim service (for duplicate detection).
3. Apply scoring rules and/or call an external API for fraud signals.
4. Determine recommended action: low risk → AUTO_APPROVE, mid risk → MANUAL_REVIEW, high risk → AUTO_REJECT.
5. Persist an analysis record for audit and explanation.
6. Return fraud score, recommended action, and triggered flags.

Internal flow — chat message

1. Receive a message from the member.
2. Load the conversation history.
3. Fetch member context: profile from IAM, active policy from Policy, recent claims from Claim.
4. Build a prompt combining context and message.
5. Call the LLM API.
6. Persist both the user message and the assistant response.
7. Return the response to the member.

Inter-service flow

- **Calls:** external LLM and OCR APIs; Policy (plan catalog and member policy); Claim (claim history); IAM (member profile for chat context)
- **Called by:** Claim (scoring, OCR, explanations); Gateway (chat, recommendations)

APIs

Method	Path	Description
`POST`	`/api/ai/claims/extract`	OCR a document and return structured fields
`POST`	`/api/ai/claims/score`	Return fraud score and recommended action
`POST`	`/api/ai/claims/explain`	Generate a plain-language decision explanation

POST	/api/ai/chat/message	Send a message to the chatbot
GET	/api/ai/chat/conversations	List the member's conversations
POST	/api/ai/policies/recommend	Recommend the best plan for user inputs

Entities

claim_analysis

Field	Description
id	Unique row identifier
claim_id	Which claim was scored
fraud_score	Risk number from 0 to 1
recommended_action	APPROVE, REJECT, or REVIEW
flags	List of reasons it flagged (duplicate, amount too high, etc.)
explanation_text	Plain-language explanation for the member
created_at	Scoring timestamp

document_extraction

Field	Description
id	Unique extraction identifier
document_id	Which uploaded file was processed
claim_id	Which claim it belongs to (nullable)
source_url	Where the file lives
extracted_fields	Structured data pulled from the document
status	PENDING, COMPLETED, or FAILED
created_at	Extraction timestamp

chat_conversation

Field	Description
id	Unique conversation identifier
member_id	Who is chatting (from IAM)
status	ACTIVE or CLOSED
last_message_at	Used for sorting most-recent first

chat_message

Field	Description
`id`	Unique message identifier
`conversation_id`	Which conversation it belongs to
`role`	USER or ASSISTANT
`content`	The actual text
`created_at`	When it was sent

plan_recommendation

Field	Description
`id`	Unique recommendation identifier
`user_id`	Who it was recommended to
`inputs_snapshot`	What the user told us (age, budget, etc.)
`recommendations`	Ranked list of plans with reasons
`created_at`	Generation timestamp

3.6 Notification service

Purpose

The Notification service sends outbound messages to members via email (and planned: SMS). It is triggered by other services when an event happens that warrants notifying the member — claim decisions, password resets, welcome emails on signup. In local development it uses MailHog to catch all outbound emails without sending real mail; in production it would use a real SMTP provider.

Internal flow

1. Receive a notification request (recipient, template name, template variables).
2. Render the template with the provided variables.
3. Send the message via SMTP (MailHog in dev, real provider in production).
4. Record the send attempt for audit purposes.
5. Return success, or queue the message for retry on failure.

Inter-service flow

- **Calls:** SMTP server (MailHog locally; real SMTP in production)
- **Called by:** Claim (decision notifications), IAM (welcome and password-reset emails)

APIs

Method	Path	Description
`POST`	`/api/notifications/email`	Send an email
`POST`	`/api/notifications/sms`	Send an SMS (future)
`GET`	`/api/notifications/me`	Get the current member's notification history

Entities

notification

Field	Description
`id`	Unique notification identifier
`recipient_id`	Who it is sent to (from IAM)
`recipient_email`	Email address used
`channel`	EMAIL or SMS
`template`	Which template was used
`subject`	Subject line
`body`	Rendered message body
`status`	PENDING, SENT, or FAILED

`sent_at`	When the message was sent
`error_message`	Failure reason if status is FAILED
`created_at`	Creation timestamp

4. Cross-cutting concerns

4.1 Authentication flow

All authenticated endpoints across the platform follow the same JWT-based flow:

1. Member registers via the IAM service (POST /api/auth/register).
2. Member logs in (POST /api/auth/login) and receives a JWT.
3. Member includes the JWT in the Authorization header on every subsequent request.
4. The Gateway validates the JWT signature and expiry before forwarding.
5. Downstream services trust the Gateway's validation; some re-validate via IAM for sensitive operations.
6. Services extract the user's role from the JWT claims for RBAC checks (member can only access their own data; staff can access all; admin has full access).

4.2 Local development

The infra repo contains a docker-compose.yml that brings up the entire platform locally:

- **PostgreSQL 16** — one container with five separate databases inside (one per service)
- **MailHog** — catches all outbound emails and shows them in a web UI
- **All six application services** — built from their respective repo folders

Steps to run locally:

1. Clone all seven repos as siblings into one workspace folder.
2. Create a `.env` file inside the infra folder with the LLM API key and any other secrets.
3. From the infra folder, run `docker-compose up -d`.
4. Access the API through the gateway at http://localhost:8080.
5. View captured emails at the MailHog UI: http://localhost:8025.

4.3 Database-per-service rule

Each service owns its own database schema. Cross-service references (a claim_id stored in the AI service's claim_analysis table, for example) are plain UUIDs — not SQL foreign keys. This is intentional: real foreign keys across service databases would couple the services at the schema level and prevent independent deployment. The trade-off is loss of referential integrity at the database level, gained back through application-level validation and event-driven consistency.

4.4 Future enhancements

The initial scope is intentionally lean. Planned additions for future iterations:

- **Kafka** — async messaging for events like claim.submitted, decoupling the Claim service from synchronous AI calls and enabling event sourcing.
- **Cloud storage** — MinIO locally or a real S3-compatible service in production for evidence file storage (currently a local Docker volume).
- **Spring Cloud Config** — centralized configuration server so services can pull their settings from one place.
- **Service discovery** — Eureka or Consul to replace hardcoded service URLs.
- **Distributed tracing** — Zipkin or Jaeger to track requests across services for debugging.
- **Centralized logging** — an ELK or Loki stack to aggregate logs from all services.
- **Real LLM integration** — connect the AI service to a real LLM provider for chatbot and explanations (currently configured but using a placeholder key).
- **Real OCR integration** — Google Document AI or AWS Textract for evidence extraction.
- **Real SMS provider** — integrate Twilio or similar for SMS notifications.
- **Admin frontend** — a web UI for staff and administrators (member-facing frontend is separate from this spec).

5. Quick reference

5.1 Service dependency map

Which services call which. Use this as a quick reference when reasoning about the impact of changes:

Service	Calls (downstream)	Called by (upstream)
Gateway	All five business services	External clients
IAM	Notification	Gateway, all other services (JWT validation)
Policy	Nothing	Claim, AI
Claim	Policy, AI, Notification	Gateway, AI
AI	Policy, Claim, IAM, external APIs	Gateway, Claim
Notification	SMTP server	Claim, IAM

5.2 Port reference

Component	Port	Notes
Gateway	8080	Public entry point
IAM	8081	Internal — accessed through gateway
Policy	8082	Internal — accessed through gateway
Claim	8083	Internal — accessed through gateway
AI	8084	Internal — accessed through gateway
Notification	8085	Internal — accessed through gateway
PostgreSQL	5432	Five databases inside one container
MailHog SMTP	1025	Receives outbound mail
MailHog UI	8025	View captured emails in browser

5.3 Database list

All five databases live inside the single PostgreSQL container, created at startup by the init script. Each is owned by one service:

- **iam** — user accounts
- **policy** — plans and member policies
- **claim** — claims and evidence metadata
- **ai** — analysis, extractions, chat history, recommendations

- **notification** — outbound message records

End of document.